

Vote → Kafka →
Worker →
PostgreSQL →
Result

Arquitectura de
Microservicios con DevOps
& Cloud Patterns

[github.com/BillyJoe121/microservices
-demo](https://github.com/BillyJoe121/microservices-demo)

MICROSERVICES DEMO

Taller I – Ingeniería de Software V

¿SOBRE QUÉ CONSTRUIMOS?

App de votación en tiempo real (Tacos vs Burritos)
) con arquitectura poliglota de microservicios
desplegados en Kubernetes / Okteto Cloud.

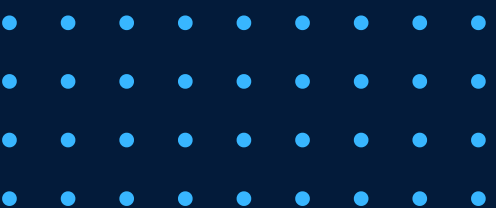
Vote Service — Java / Spring Boot

Worker — Go

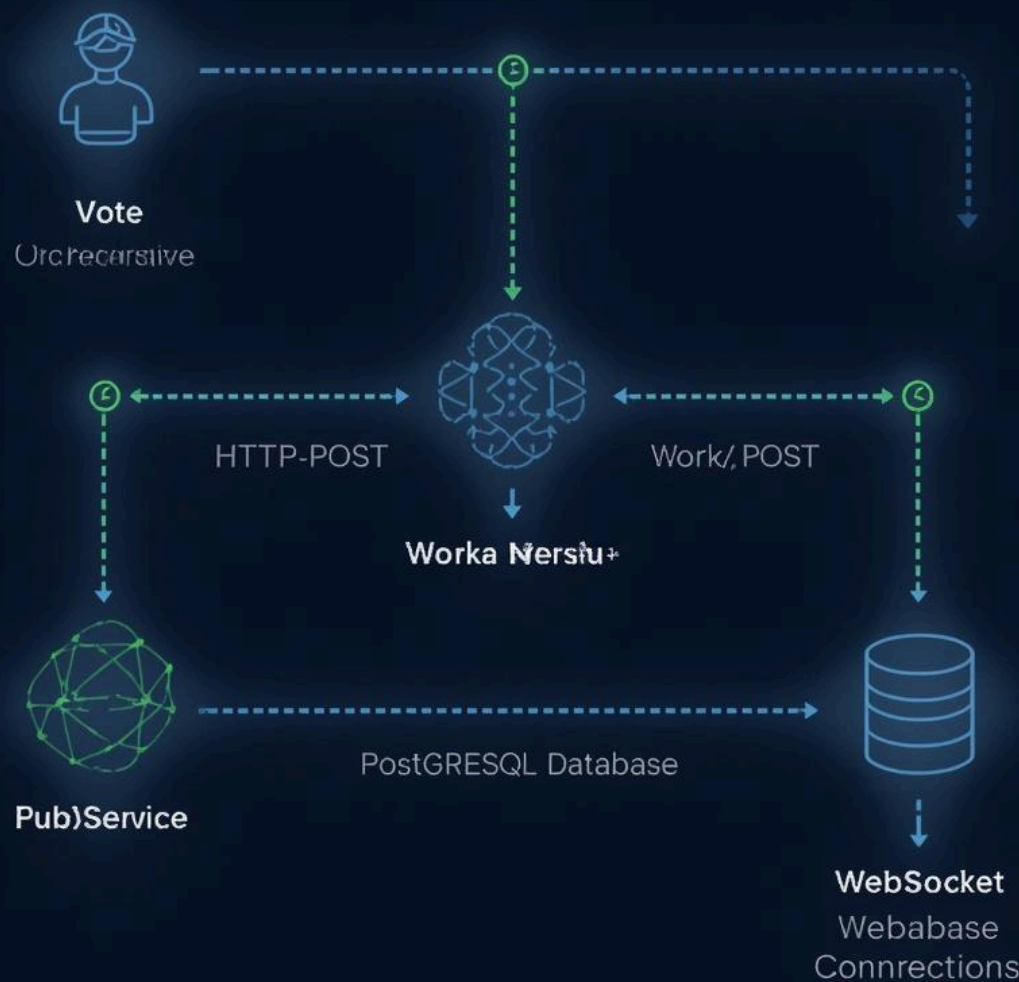
Result — Node.js / Express

PostgreSQL

Apache Kafka



ARQUITECTURA DEL SISTEMA



Flujo Asíncrono

Vote publica en Kafka sin bloquear respuesta.

Tiempo Real

WebSocket push instantáneo de resultados al usuario.

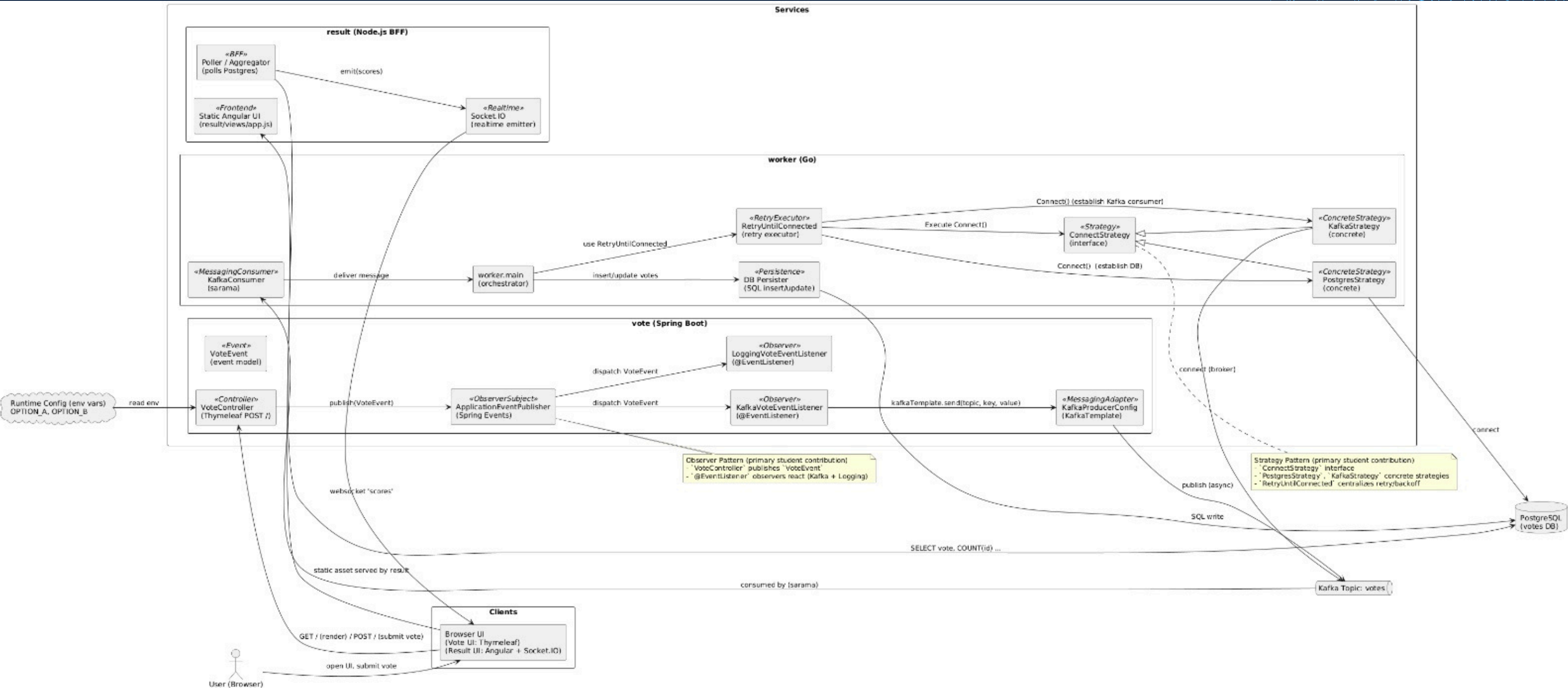
Desacoplado

Servicios independientes comunicados por mensajería.

Socket.io

Conexión bidireccional cliente-servidor.

ARQUITECTURA DEL SISTEMA



3 PATRONES DE DISEÑO DE NUBE APLICADOS



Async Messaging

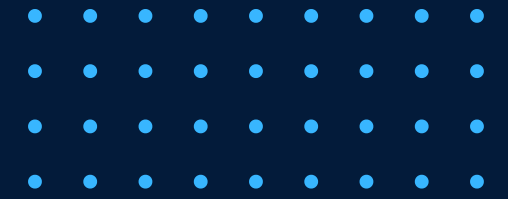


External
Config



Backends for FE

2 PATRONES DE SOFTWARE IMPLEMENTADOS



Strategy

Java — Observer Pattern
(rama feature/vote-observer-pattern)

Antes: loops duplicados
Ahora: función genérica e intercambiable



Observer

```
getHandlers().forEach(handler ->  
  handler.onVoteReceived(voter, vote));  
// KafkaVoteHandler, LoggingVoteHandler
```

Antes: controlador hacía todo
Ahora: notifica observadores registrados

GITHUB FLOW – MAIN SIEMPRE DESPLEGABLE

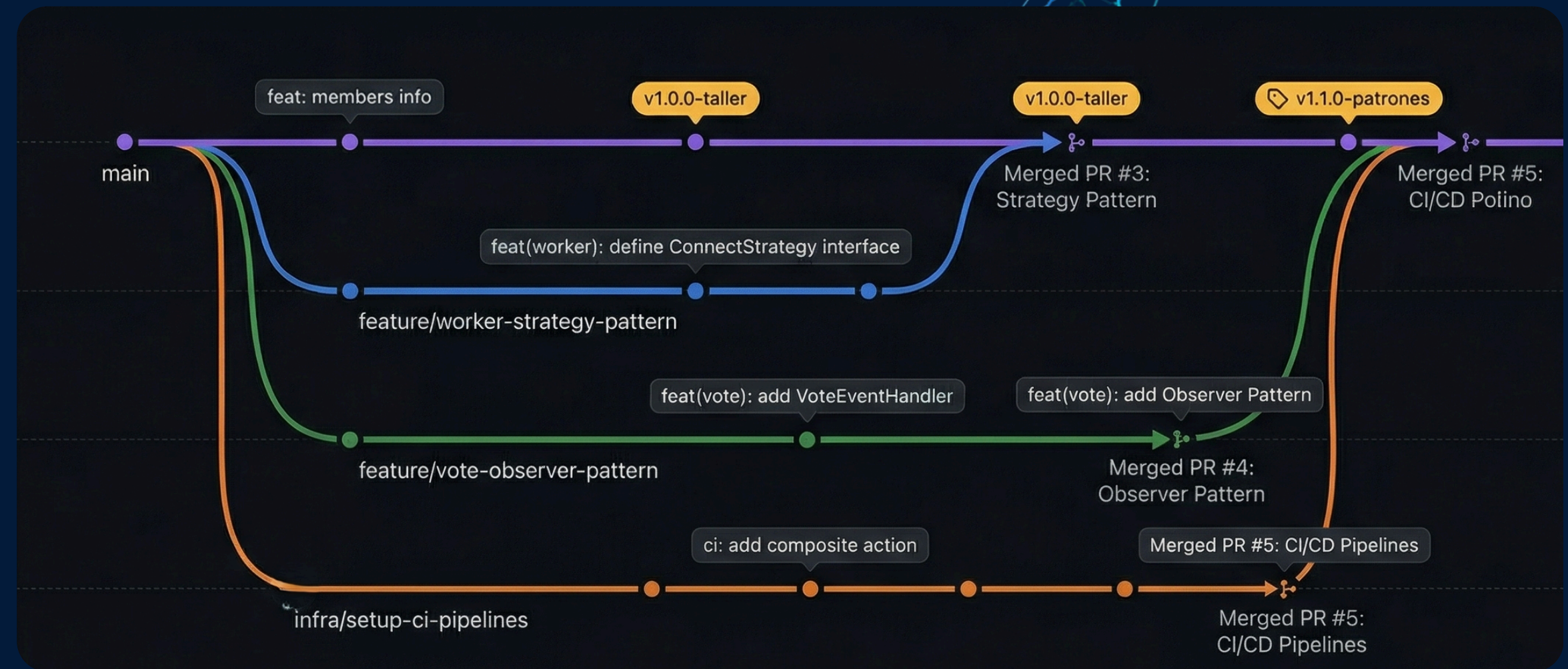
Cada merge vía PR
+ Code Review
+ CI Verde

Ramas:

- main (protegida)
- feature/
- infra/
- hotfix/

Tags semánticos:

- v1.0.0-taller
- v1.1.0-patrones





3 PIPELINES CI INDEPENDIENTES + 1 CD UNIFICADO



Path Filtering

Se activa solo si cambia
su microservicio (paths:
vote/**).



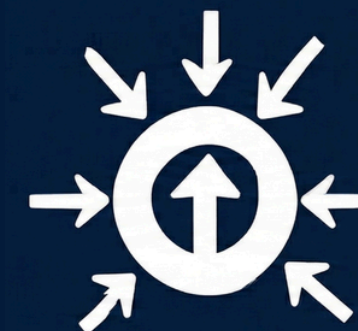
Composite Action

Build + Scan en
archivo reutilizable
(DRY).



Shift-Left Security

Trivy Scan rechaza
imagen si hay CVE
crítica.



CD Unificado

staging →
production con
aprobación.

```
docker-compose.yml
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.4.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000
    ports:
      - '2181:2181'
    networks:
      - micro-demo

  kafka:
    image: confluentinc/cp-kafka:7.4.0
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    ports:
      - '9092:9092'
    depends_on:
      - zookeeper
    networks:
      - micro-demo

  postgresql:
    image: postgres:13
    environment:
      - POSTGRES_USER=okteto
      - POSTGRES_PASSWORD=okteto
      - POSTGRES_DB=votes
    ports:
      - '5435:5432'
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
```

INFRASTRUCTURE.YML – IAC COMO CÓDIGO DE PRIMERA CLASE



yamllint

Sintaxis y estructura YAML
validadas
automáticamente.



helm lint

Validación de charts Kafka +
PostgreSQL.



hadolint

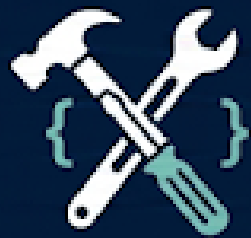
Anti-patrones en Dockerfiles
detectados.



Checkov

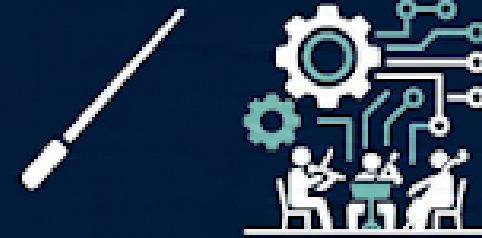
Escaneo IaC vs políticas CIS.

INFRAESTRUCTURA ZERO TRUST



Desarrollo

`docker-compose.yml` ·
`okteto.yml` para entornos
locales y cloud dev.



Orquestación

Kustomize (apps
propias) · Helm (Kafka,
PostgreSQL terceros).



Seguridad

`runAsUser:10001` ·
`readOnlyRootFilesystem` ·
`NetworkPolicy default-`
`deny`.



Confiabilidad

`livenessProbe` ·
`readinessProbe` ·
`PersistentVolumeClai`
`m`.

¿QUÉ LOGRAMOS?



5 Pipelines — Vote·Worker·Result·CD·Infra



0 Violaciones — Checkov+Yamllint en main



5 Patrones — 3 Nube + 2 Software



3 Lenguajes — Java·Go·Node.js



GitHub Flow — PRs obligatorios · Conventional Commits



DevSecOps — Trivy + Checkov Shift-Left



DRY via Composite Actions



Zero Trust: 100-root, filesystem immutable, default-deny network

"No solo desplegamos pods. Los desplegamos de forma segura, automatizada y con calidad industrial."
William Joseph Verdesoto . Juan Camilo Corrales Osvath | github.com/BillyJoel21/microsorie-demo

DEMO